Lập trình mạng · Chương 4

Web service: REST, tài nguyên, phương thức HTTP và JSON

TS. Đặng Ngọc Hùng

REST dùng chính URI, phương thức và mã trạng thái của HTTP

1. SOAP: mọi thao tác là một **POST** tới cùng một địa chỉ; thao tác, tham số, lỗi đều nằm trong SOAP envelope
2. HTTP đã có sẵn: **URI** gọi tên đối tượng, **phương thức** nói thao tác, **mã trạng thái** báo kết quả
3. REST dùng chính các thành phần đó; thân message thường là **JSON**

> **Câu hỏi của phần:** thiết kế API REST thế nào, và client xử lý kết quả ra sao?

Mỗi con vật là một tài nguyên; phương thức là thao tác, mã trạng thái là kết quả

`http://127.0.0.1:9199`**`/pet/10`**

endpoint của server — tài nguyên: con vật số 10 trong tập pet

request	thao tác	mã trạng thái	idempotent
`GET /pet/{petId}`	đọc một con vật	200 kèm JSON · 404 không có · 400 id sai	có (không đổi dữ liệu)
`POST /pet + JSON`	thêm vào tập, server cấp id	201 kèm Location: /pet/11	không (mỗi lần thêm một con)
`PUT /pet/{petId} + JSON`	thay toàn bộ con vật	200 đã thay · 404 không có	có (ghi đè cùng giá trị)
`DELETE /pet/{petId}`	xóa con vật	204 đã xóa · 404 không có	có (vẫn là không có)
phương thức khác	không hỗ trợ	405 kèm Allow · JSON sai: 400	

> **REST**: URI định danh **tài nguyên**, **phương thức HTTP** là thao tác, **mã trạng thái HTTP** báo kết quả; server **stateless**.

SOAP thiết kế quanh thao tác, REST thiết kế quanh tài nguyên

	SOAP · quanh thao tác	REST · quanh tài nguyên
request	`POST /tempconvert` thao tác, tham số trong envelope XML: 242 byte	`GET /pet/10` thao tác là phương thức, tài nguyên là URI, không có thân
response	`200 · text/xml` envelope XML: 259 byte cho một con số	`200 · application/json` JSON: 46 byte cho cả một con vật
báo lỗi	nằm trong Body (SOAP Fault)	mã trạng thái HTTP: 404 không có, 400 yêu cầu sai
hợp đồng	bắt buộc: WSDL	không bắt buộc; Petstore là ví dụ mẫu của OpenAPI

> REST có message nhỏ hơn và dùng chính HTTP; đổi lại, hợp đồng không bắt buộc.

Server REST bằng HttpServer: URI cho tài nguyên, phương thức cho thao tác

```java
server.createContext("/pet", RestServer::handle);  // tập /pet và từng con vật /pet/{petId}

long id = NEXT_ID.getAndIncrement();  // POST /pet: server cấp id, mỗi POST thêm một con vật mới
ex.getResponseHeaders().set("Location", "/pet/" + id);  // URI của tài nguyên vừa tạo

long id = Long.parseLong(path.substring("/pet/".length()));  // URI định danh tài nguyên
switch (method) {  // phương thức HTTP là thao tác
    case "GET" -> {
        Map<String, Object> pet = PETS.get(id);
```

- `createContext`: mọi request có URI bắt đầu bằng /pet tới cùng một handler
- POST vào tập: server **cấp id**, trả 201 kèm header **Location**
- Với một con vật: id lấy từ **URI**, thao tác chọn theo **phương thức HTTP**, mỗi nhánh trả **mã trạng thái** kèm **JSON**

Client HttpClient: đọc mã trạng thái trước, rồi mới đọc JSON

Java

```java
HttpRequest request = HttpRequest.newBuilder(URI.create(BASE + path))
        .method(method, json == null ? BodyPublishers.noBody() : BodyPublishers.ofString(json))
        .header("Content-Type", "application/json")
        .build();
HttpResponse<String> response = CLIENT.send(request, BodyHandlers.ofString());
int status = response.statusCode();  // đọc mã trạng thái trước, rồi mới đọc thân
```

Kết quả chạy

```
GET    /pet/10  -> 200 {"id":10,"name":"doggie","status":"available"}
GET    /pet/99  -> 404 {"message":"Pet not found"}
GET    /pet/abc -> 400 {"message":"Invalid input"}
POST   /pet     -> 201 {"id":11,"name":"kitty","status":"available"}
POST   /pet     -> 201 {"id":12,"name":"kitty","status":"available"}
PUT    /pet/11  -> 200 {"id":11,"name":"kitty","status":"sold"}
PUT    /pet/11  -> 200 {"id":11,"name":"kitty","status":"sold"}
DELETE /pet/12  -> 204
DELETE /pet/12  -> 404 {"message":"Pet not found"}
PATCH  /pet/11  -> 405 {"message":"Method not allowed"}
```

Báo lỗi bằng mã trạng thái; chỉ tự gọi lại phương thức idempotent

1. Trả 200 kèm thân báo lỗi: client và cache coi là **thành công**
2. **PUT, DELETE** idempotent: gọi lại để lại cùng trạng thái, dù mã trạng thái có thể khác (204 rồi 404)
3. **POST** không idempotent: hai lần gọi tạo **hai con vật**

> **Nguyên tắc:** client chỉ tự gọi lại các phương thức idempotent: GET, PUT, DELETE.

Câu hỏi 1

Với con vật không tồn tại, server REST trả mã 200 kèm thân {"message": "Pet not found"}. Điều gì có thể sai?

A. Client chỉ kiểm tra mã trạng thái và cache trên đường đi đều coi đây là kết quả thành công.

B. Không sai gì, vì thân JSON đã ghi rõ lỗi.

C. HttpClient ném ngoại lệ vì thân không đúng định dạng.

D. Server phải chuyển sang SOAP để báo được lỗi.

Câu hỏi 1

Với con vật không tồn tại, server REST trả mã 200 kèm thân {"message": "Pet not found"}. Điều gì có thể sai?

A. Client chỉ kiểm tra mã trạng thái và cache trên đường đi đều coi đây là kết quả thành công.

B. Không sai gì, vì thân JSON đã ghi rõ lỗi.

C. HttpClient ném ngoại lệ vì thân không đúng định dạng.

D. Server phải chuyển sang SOAP để báo được lỗi.

Mã trạng thái là cách HTTP báo kết quả cho mọi thành phần trên đường đi. Với tài nguyên không tồn tại, mã đúng là 404; trả 200 khiến client và cache hiểu là thành công.

Câu hỏi 2

Client gửi request nhưng kết nối hỏng trước khi nhận được response. Với những phương thức nào, client có thể tự gọi lại mà không cần cơ chế bổ sung?

A. POST và PUT.

B. Chỉ POST.

C. GET, PUT và DELETE.

D. Mọi phương thức, vì HTTP chạy trên TCP.

Câu hỏi 2

Client gửi request nhưng kết nối hỏng trước khi nhận được response. Với những phương thức nào, client có thể tự gọi lại mà không cần cơ chế bổ sung?

A. POST và PUT.

B. Chỉ POST.

C. GET, PUT và DELETE.

D. Mọi phương thức, vì HTTP chạy trên TCP.

GET, PUT, DELETE là idempotent: thực hiện lại để lại cùng trạng thái. POST không idempotent, gọi lại cần idempotency key.

Tổng kết

1. **REST**: **tài nguyên** định danh bằng **URI**; **phương thức HTTP** là thao tác; **mã trạng thái HTTP** báo kết quả
2. Thân message thường là **JSON**; server **stateless**, mỗi request mang đủ thông tin
3. GET, PUT, DELETE idempotent, POST thì không; báo lỗi bằng mã trạng thái

Từ khóa cần nhớ REST · tài nguyên · URI · phương thức HTTP · mã trạng thái HTTP · JSON · stateless

Mở rộng

Chủ đề nâng cao

- OpenAPI: mô tả hợp đồng cho API REST
- Quản lý phiên bản API

Bài tập mở rộng

Thêm GET /pet/findByStatus?status=available như đặc tả Petstore (tham số truy vấn, trả mảng JSON); đối chiếu API với tệp OpenAPI của Petstore công khai (https://petstore3.swagger.io).

Tiếp theo

gRPC: Protocol Buffers và lời gọi unary